

Configuración de Servidor Jenkins e Integración con Kubernetes

Este documento detalla los pasos para desplegar Jenkins en un contenedor Docker e integrarlo con un clúster de Kubernetes (como MicroK8s).

1. Despliegue de Jenkins en Docker

Para iniciar el contenedor de Jenkins permitiendo la comunicación con el host de Kubernetes:

```
docker run -d -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  --add-host=host.docker.internal:host-gateway \
  --name jenkins jenkins/jenkins:lts
```

2. Preparación del Entorno en el Contenedor

Acceda al contenedor como usuario `root` para instalar las herramientas necesarias:

```
docker exec -it --user=root jenkins /bin/bash
```

Instalación de `kubectl`

Dentro del contenedor, ejecute los siguientes comandos:

```
curl -fsSL "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" -o
/usr/local/bin/kubectl
chmod +x /usr/local/bin/kubectl
```

3. Configuración de Acceso al Clúster

Obtención del Token de Autenticación

Desde la terminal de su máquina host (donde corre el clúster), obtenga el token generado para Jenkins:

```
kubectl get secret jenkins-token -o jsonpath='{.data.token}' | base64 --decode
```

Validación de Conexión desde Jenkins

Dentro del contenedor de Jenkins, verifique que puede alcanzar el API Server:

Prueba de red con curl:

```
curl -k -H "Authorization: Bearer <TOKEN>" https://host.docker.internal:16443/livez
```

Prueba de consulta al clúster:

```
kubectl --server=https://host.docker.internal:16443 \
  --token=<TOKEN> \
  --insecure-skip-tls-verify \
  get nodes
```

4. Configuración en la Interfaz Web de Jenkins

1. **Instalación de Plugins:** Instale `Kubernetes Plugin` y, de forma recomendada, `Kubernetes CLI`.
2. **Agregar Credenciales:**
 - Vaya a: **Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials**.
 - **Kind:** Secret text.
 - **Secret:** Pegue el token obtenido anteriormente.
 - **ID:** `k8s-token` (o el nombre que prefiera para su pipeline).

Aplicación de Roles y Objetos de Kubernetes

Para que Jenkins pueda gestionar recursos, debemos aplicar la configuración de RBAC definida en `jenkins-account.yaml`. A continuación se detallan los objetos utilizados:

```
kubectl apply -f jenkins-account.yaml
```

Desglose de Objetos RBAC

1. ServiceAccount

- **Definición:** Proporciona una identidad para los procesos que se ejecutan en un Pod o para entidades externas que interactúan con el API.
- **Documentación:** [Service Accounts](#)
- **Función:** Actúa como el "usuario" que Jenkins utilizará para identificarse ante Kubernetes.

2. Role

- **Definición:** Define un conjunto de permisos (reglas) que permiten realizar acciones sobre recursos dentro de un **namespace específico**.
- **Documentación:** [RBAC Roles](#)
- **Función:** Especifica qué puede hacer Jenkins (crear pods, listar servicios, ver logs, etc.) exclusivamente en el namespace `default`.

3. RoleBinding

- **Definición:** Vincula un `Role` a un sujeto (en este caso, al `ServiceAccount`).
- **Documentación:** [RBAC RoleBinding](#)
- **Función:** Otorga de manera efectiva los permisos del `Role` al `ServiceAccount` de Jenkins dentro del namespace correspondiente.

4. ClusterRole

- **Definición:** Similar a un `Role`, pero define permisos a nivel de **todo el clúster** o sobre recursos no asociados a namespaces.
- **Documentación:** [RBAC ClusterRole](#)
- **Función:** Permite definir permisos sobre recursos globales como `PersistentVolumes` (PV), necesarios si Jenkins debe aprovisionar almacenamiento.

5. ClusterRoleBinding

- **Definición:** Vincula un `ClusterRole` a un sujeto a nivel de todo el clúster.
- **Documentación:** [RBAC ClusterRoleBinding](#)

- **Función:** Permite que Jenkins gestione recursos globales (como los PV mencionados) sin importar el namespace.

6. Secret

- **Definición:** Objeto diseñado para almacenar datos sensibles, como contraseñas o tokens.
- **Documentación:** [Secrets](#)
- **Función:** Desde Kubernetes 1.24, los tokens no se crean automáticamente. Este objeto genera un **token de larga duración** vinculado al `ServiceAccount` para que Jenkins pueda autenticarse de forma persistente.

```

#this file define a service account for kubernetesplugins on jenkins
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: default
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: jenkins
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods","services"]
  verbs: ["create","delete","get","list","patch","update","watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["create","delete","get","list","patch","update","watch"]
- apiGroups: [""]
  resources: ["pods/exec"]
  verbs: ["create","delete","get","list","patch","update","watch"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get","list","watch"]
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get"]
- apiGroups: [""]
  resources: ["persistentvolumeclaims"]
  verbs: ["create","delete","get","list","patch","update","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: jenkins
subjects:
- kind: ServiceAccount
  name: jenkins
---

```

```

# Allows jenkins to create persistent volumes
# This cluster role binding allows anyone in the "manager" group to read secrets in
any namespace.
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: jenkins-crb
subjects:
- kind: ServiceAccount
  namespace: default
  name: jenkins
roleRef:
  kind: ClusterRole
  name: jenkinsclusterrole
  apiGroup: rbac.authorization.k8s.io
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  # "namespace" omitted since ClusterRoles are not namespaced
  name: jenkinsclusterrole
rules:
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
---
# https://kubernetes.io/docs/concepts/configuration/secret/
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: jenkins-token
  namespace: default
  annotations:
    kubernetes.io/service-account.name: "jenkins"
---

```

Validación Mediante Pipeline

Utilice este ejemplo de `Jenkinsfile` para confirmar la integración total:

```

pipeline {
    agent any

    environment {
        K8S_URL = "https://host.docker.internal:16443"
        K8S_CRED_ID = "k8s-token"
    }

    stages {
        stage('Validate K8s Connection') {
            steps {
                script {
                    withKubeConfig([credentialsId: env.K8S_CRED_ID, serverUrl:
env.K8S_URL]) {
                        echo "--- Probando conexión al clúster ---"
                        sh 'kubectl cluster-info'

                        echo "--- Listando nodos ---"
                        sh 'kubectl get nodes'

                        echo "--- Verificando permisos de despliegue ---"
                        sh 'kubectl auth can-i create deployments'
                    }
                }
            }
        }
    }
}

```